Network File System (NFS) Implementation

Course Project - Hot Single Threads Near You

Review the assignment due date

Table of Contents

1. Overview
2. Architecture
3. File Structure Implementation
4. Features
5. Directory Structure
6. Compilation
7. Usage
8. Commands
9. Implementation Details
10. Assumptions
11. Design Decisions

Overview

This is a distributed Network File System (NFS) implementation in C that allows multiple clients to perform file operations across multiple storage servers coordinated by a central Name Server. The system supports concurrent access, access control, file streaming, command execution, and efficient file search capabilities.

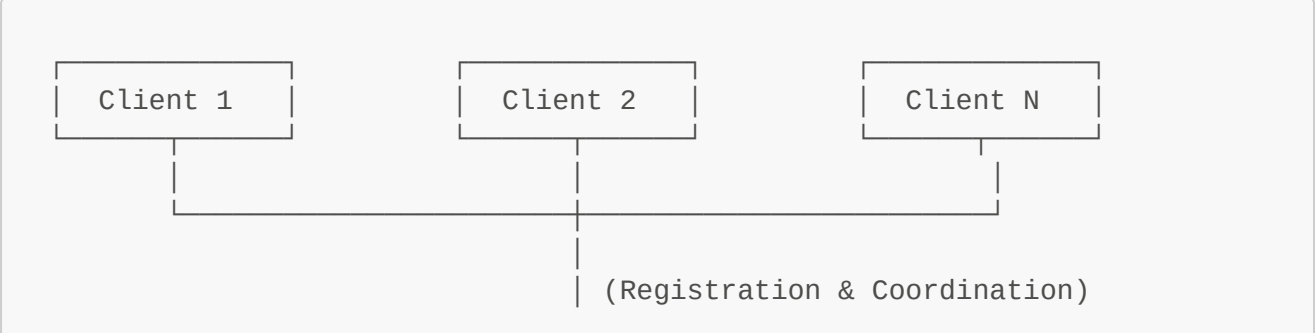
Key Components

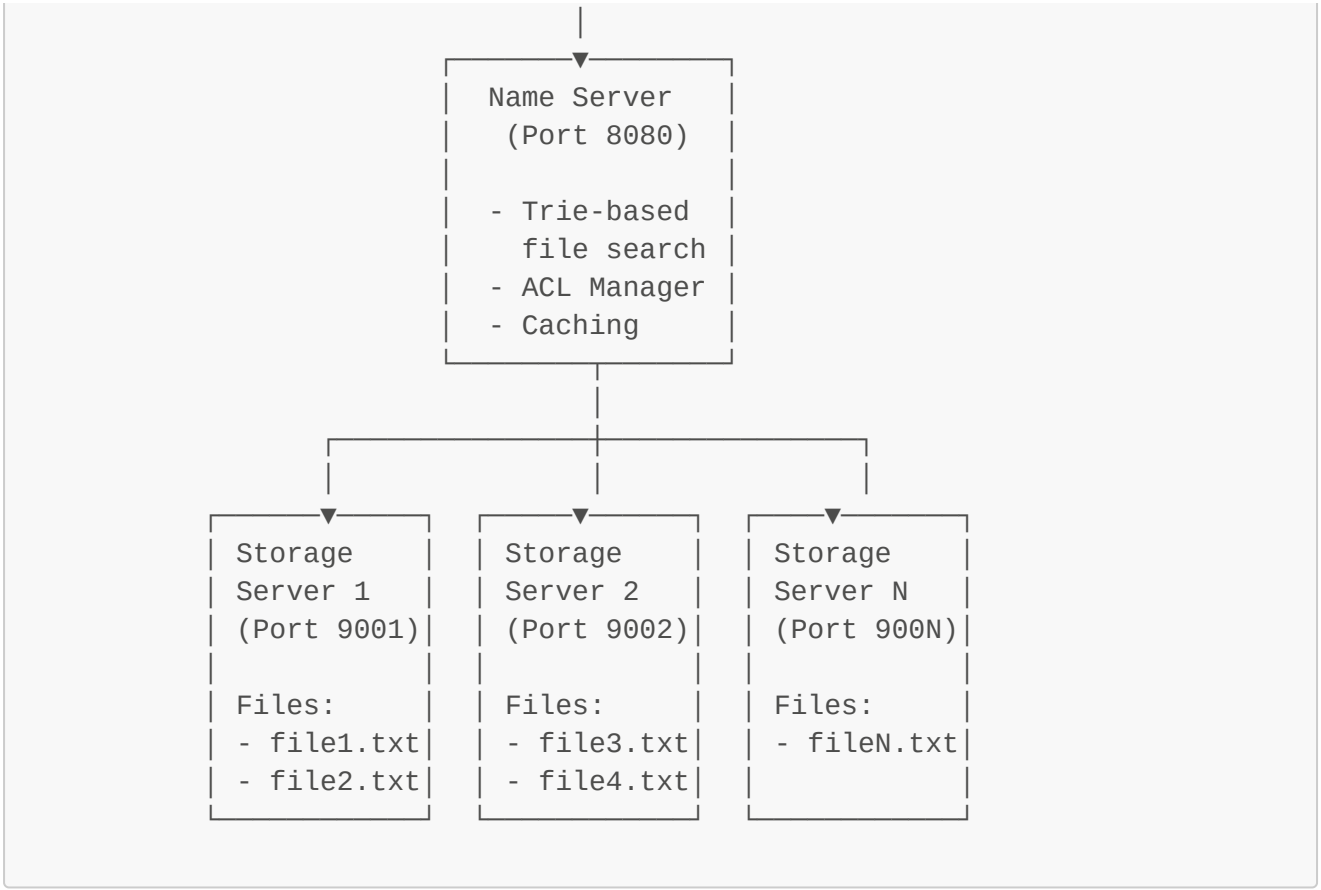
PROF

1. **Name Server (NM)**: Central coordinator that manages file locations, access control, and routes client requests to appropriate storage servers
2. **Storage Servers (SS)**: Store and manage files with support for concurrent read/write operations
3. **Clients**: User interfaces that connect to the Name Server to perform file operations

---

Architecture





Communication Flow

1. **Registration Phase**:
    - Storage Servers connect to Name Server and register with their file lists
    - Clients connect to Name Server and register with their username
2. **File Operations**:
    - Client sends request to Name Server
    - Name Server validates permissions and locates the appropriate Storage Server
    - For READ/WRITE/STREAM operations: Name Server returns SS location, client connects directly
    - For CREATE/DELETE operations: Name Server forwards request to SS
    - For INFO/VIEW/LIST operations: Name Server handles directly

PROF

File Structure Implementation

2D Linked List Design

Files are implemented as a **2D linked list** structure:

```
File Structure
      |
      ├─→ Sentence 1 ──→ Word1 ──→ Word2 ──→ Word3 ──→ NULL
      |
```

```
        ├─→ Sentence 2 ──→ Word1 ──→ Word2 ──→ NULL
        │
        ├─→ Sentence 3 ──→ Word1 ──→ Word2 ──→ Word4 ──→ Word5 ──→ NULL
        │
        └─→ NULL
```

Data Structures

```c
// Word Node - represents a single word
typedef struct WordNode {
    char word[MAX_WORD_LENGTH];
    struct WordNode* next;
} WordNode;

// Sentence Node - represents a sentence (linked list of words)
typedef struct SentenceNode {
    WordNode* words;
    int word_count;
    struct SentenceNode* next;
} SentenceNode;

// File Structure - represents entire file (linked list of sentences)
typedef struct FileStructure {
    SentenceNode* sentences;
    int sentence_count;
    int total_words;
    int total_chars;
} FileStructure;
```

Benefits

PROF

1. **Efficient Sentence-Level Operations**: Direct access to specific sentences for locking and editing
2. **Word-Level Granularity**: Easy to insert, update, or delete individual words
3. **Dynamic Size**: No fixed limits on file size, sentences, or words
4. **Memory Efficient**: Only allocates memory for actual content
5. **Easy Sentence Parsing**: Natural fit for sentence-based editing with sentence delimiters (. ! ?)

Features

Implemented Features

User Functionalities

- ✅ **[10] VIEW**: List files with flags `-a` (all), `-l` (detailed), `-al` (both)
- ✅ **[10] READ**: Read complete file contents
- ✅ **[10] CREATE**: Create new empty files

- ✅ **[30] WRITE**: Sentence-level editing with word-level updates
  - Supports sentence locking
  - Handles sentence delimiters (. ! ?)
  - Multiple word updates per session
- ✅ **[15] UNDO**: Revert last change (file-specific, up to 10 versions)
- ✅ **[10] INFO**: File metadata (size, word count, character count, timestamps)
- ✅ **[10] DELETE**: File deletion by owner
- ✅ **[15] STREAM**: Word-by-word streaming with 0.1s delay
- ✅ **[10] LIST**: Display all registered users
- ✅ **[15] ACCESS CONTROL**: Read/Write permissions (ADDACCESS, REMACCESS)
- ✅ **[15] EXEC**: Execute file content as shell commands on Name Server

System Requirements

- ✅ **[10] Data Persistence**: Files and metadata stored on disk
- ✅ **[5] Access Control**: ACL-based permissions
- ✅ **[5] Logging**: Comprehensive logging with timestamps, IPs, ports
- ✅ **[5] Error Handling**: Universal error codes across system
- ✅ **[15] Efficient Search**: O(m) Trie-based file search + LRU caching

---

Directory Structure

```
course-project-hot-single-threads-near-you/
├── README.md                          # This file
├── Makefile                           # Build system
├── include/
│   ├── common.h                       # Common definitions, structures,
constants
│   └── file_structure.h               # 2D linked list interface
├── src/
│   ├── common/
│   │   ├── common.c                   # Network utilities, logging
│   │   └── file_structure.c           # 2D linked list implementation
│   ├── name_server/
│   │   └── name_server.c              # Name Server implementation
│   ├── storage_server/
│   │   └── storage_server.c           # Storage Server implementation
│   └── client/
│       └── client.c                   # Client implementation
├── storage/                           # Storage server data directories
│   ├── ss1/                          # Storage Server 1 files
│   ├── ss2/                          # Storage Server 2 files
│   └── ...
├── logs/                              # Log files
│   ├── name_server.log
│   ├── storage_server.log
│   └── client.log
```

PROF

```
└── build/                              # Compiled object files
(generated)
```

Compilation

Prerequisites

- GCC compiler
- POSIX threads library (pthread)
- GNU Readline library
- Linux/Unix environment

Install Dependencies (Ubuntu/Debian)

```
sudo apt-get update
sudo apt-get install build-essential libreadline-dev
```

Build All Components

```
make
```

This will compile:

- `name_server` - Name Server executable
- `storage_server` - Storage Server executable
- `client` - Client executable

PROF

Build Individual Components

```
make name_server      # Build Name Server only
make storage_server   # Build Storage Server only
make client          # Build Client only
```

Clean Build

```
make clean           # Remove build artifacts
make clean-all       # Remove build artifacts + logs + storage
```

Usage

Quick Start (Automated)

Start Name Server and 2 Storage Servers automatically:

```
make run-servers
```

In another terminal, start a client:

```
make run-client
```

Stop all servers:

```
make stop-servers
```

Manual Start

1. Start Name Server

```
./name_server
```

The Name Server runs on port 8080 by default.

2. Start Storage Servers

In separate terminals:

PROF

```
# Storage Server 1
./storage_server 1 9001

# Storage Server 2
./storage_server 2 9002

# Storage Server N
./storage_server N 900N
```

Parameters:

- First argument: Storage Server ID (unique integer)
- Second argument: Client port (port for client connections)

3. Start Clients

In separate terminals:

```
./client
```

When prompted, enter a username. Each client can have a different username for access control.

BONUS FEATURES

1. Hierarchical Folder Structure (10 points)

The NFS now supports organizing files into hierarchical folders, allowing users to create folder structures and move files between folders.

Commands:

- `CREATEFOLDER <folder_path>` - Create a new folder
- `MOVE <filename> <folder_path>` - Move a file to a folder
- `VIEWFOLDER [folder_path]` - View contents of a folder (defaults to root "/")

Example:

```
CREATEFOLDER /project/docs
CREATE report.txt
MOVE report.txt /project/docs
VIEWFOLDER /project/docs
```

Implementation Details:

- Folders are tracked in the Name Server with owner and creation timestamps
- Files maintain a `folder_path` field indicating their location
- VIEWFOLDER displays both subfolders and files with owner information

PROF

2. Checkpoint Mechanism (15 points)

Users can save checkpoints (snapshots) of files and revert to previous versions when needed.

Commands:

- `CHECKPOINT <filename> <tag>` - Save a checkpoint with a tag
- `VIEWCHECKPOINT <filename> <tag>` - View checkpoint content without modifying the file
- `REVERT <filename> <tag>` - Revert file to a previous checkpoint
- `LISTCHECKPOINTS <filename>` - List all available checkpoints for a file

Example:

```
```

```
CHECKPOINT report.txt v1
# ... make changes ...
CHECKPOINT report.txt v2
LISTCHECKPOINTS report.txt
VIEWCHECKPOINT report.txt v1
REVERT report.txt v1
```

Implementation Details:

- Checkpoints are stored as hidden files on the Storage Server: `.checkpoint_<filename>_<tag>`
- Maximum 20 checkpoints per file (configurable via MAX_CHECKPOINTS)
- Current version is saved to undo stack before reverting
- Checkpoints persist across sessions

3. Access Request System (5 points)

Users can request access to files they don't own, and file owners can approve or deny these requests.

Commands:

- `REQUESTACCESS <filename> <READ|WRITE>` - Request read or write access
- `LISTREQUESTS` - View pending access requests for files you own
- `APPROVE <filename> <requester_username>` - Grant access to a user
- `DENY <filename> <requester_username>` - Deny access request

Example:

```
# User alice requests access to bob's file
REQUESTACCESS report.txt READ

# Bob (owner) views and processes requests
LISTREQUESTS
APPROVE report.txt alice

# Alice can now read the file
READ report.txt
```

PROF

Implementation Details:

- Access requests are stored in a linked list in the Name Server
- LISTREQUESTS only shows requests for files owned by the current user
- APPROVE automatically adds the user to the file's ACL
- Maximum 100 pending requests (configurable via MAX_ACCESS_REQUESTS)

Commands

File Viewing

```
VIEW           # List files you have access to
VIEW -a        # List all files on the system
VIEW -l        # List files with details (word count, char count, owner,
etc.)
VIEW -al       # List all files with details
```

File Operations

```
CREATE <filename>                    # Create new file
READ <filename>                      # Read file content
DELETE <filename>                    # Delete file (owner only)
INFO <filename>                      # Display file metadata
```

File Editing

```
WRITE <filename> <sentence_number>   # Start editing a sentence
# Then enter updates as:
<word_index> <content>
<word_index> <content>
...
ETIRW                               # Finish editing

# Example:
WRITE myfile.txt 1
0 Hello
1 world
2 !
ETIRW
```

PROF

Important:

- Word indices start at 0
- Content can contain sentence delimiters (. ! ?) which create new sentences
- The sentence is locked during editing

```
UNDO <filename>                      # Undo last change to file
```

Streaming

```

```

```
STREAM <filename>                    # Stream file word-by-word (0.1s
delay)
```

User Management

```
LIST                                 # List all users in system
```

Access Control

```
ADDACCESS -R <filename> <username>   # Grant read access
ADDACCESS -W <filename> <username>   # Grant write (and read) access
REMACCESS <filename> <username>      # Remove all access
```

Note: Only file owner can manage access. Owner always has full permissions.

Command Execution

```
EXEC <filename>                      # Execute file content as shell
commands
```

The file content is executed on the Name Server, and output is returned to client.

Client Control

```
HELP                                 # Show available commands
EXIT                                 # Disconnect from server
```

Implementation Details

Name Server

Efficient File Search (O(m) complexity)

Uses a **Trie data structure** for file name lookup:

```
typedef struct TrieNode {
    struct TrieNode* children[128];  // ASCII character set
    int is_end_of_word;
```

```
    int file_index;
} TrieNode;
```

- **Search Complexity**: O(m) where m = filename length
- Much better than linear O(N) search through all files
- Constant time character lookup

Caching System

LRU-style cache for recent file lookups:

```
typedef struct CacheEntry {
    char filename[MAX_FILENAME];
    int ss_id;
    time_t timestamp;
    int valid;
} CacheEntry;
```

- Cache size: 100 entries
- Cache timeout: 60 seconds
- Reduces lookup time for frequently accessed files

Access Control

Access Control Lists (ACL) stored per file:

```
typedef struct AccessEntry {
    char username[MAX_USERNAME];
    int permissions;  // ACCESS_READ or ACCESS_WRITE
    struct AccessEntry* next;
} AccessEntry;
```

PROF

- Owner: Always has full access
- Read permission: Can VIEW, READ, INFO, STREAM, EXEC
- Write permission: Includes read + can WRITE, DELETE

Thread Safety

- Separate mutexes for storage servers, clients, files, and cache
- Prevents race conditions in multi-client scenarios
- Each file has its own lock for metadata updates

Storage Server

File Persistence

Files stored in `storage/ssN/` directories:

- Plain text format for easy debugging
- Loaded into 2D linked list for operations
- Saved back to disk after modifications

Undo System

```
FileStructure* undo_stack[MAX_FILES][MAX_UNDO_STACK];
int undo_top[MAX_FILES];
```

- Per-file undo history (up to 10 versions)
- Stores complete file structure snapshots
- File-level undo (not user-specific)

Sentence Locking

```
int sentence_locks[MAX_FILES][1000];  // Boolean locks per sentence
```

- Prevents concurrent edits to same sentence
- Multiple users can edit different sentences simultaneously
- Automatically released after WRITE completes

Concurrent Access

- Multi-threaded design with one thread per client connection
- File-level locks using pthread_mutex
- Separate server sockets for Name Server and Client connections

PROF

Client

Interactive Interface

Uses GNU Readline for:

- Command history (up/down arrows)
- Command line editing
- Tab completion (future enhancement)

Direct Storage Server Connection

For READ/WRITE/STREAM operations:

1. Client requests SS location from Name Server
2. Name Server returns IP and port

3. Client connects directly to Storage Server
4. Reduces Name Server load for data transfer

Assumptions

Network

1. All components run on localhost (127.0.0.1)
2. Name Server uses fixed port 8080
3. Storage Servers use ports 9000+
4. No network partitions or failures (except graceful disconnects)

Files

1. Files contain only text (ASCII) data
2. Every period (.), exclamation (!), or question (?) is a sentence delimiter
3. No size limit on files, but practical limit due to MAX_BUFFER_SIZE for network packets
4. File names are unique across the system

Concurrency

1. Multiple clients can connect simultaneously
2. Multiple storage servers can be active
3. Only one Name Server instance runs at a time
4. If Name Server fails, entire system must restart

Access Control

1. Username-based authentication (no passwords)
2. First user to create a file becomes owner
3. Owner cannot transfer ownership
4. System user owns files created at startup

PROF

Sentence Parsing

1. Words are sequences of non-space ASCII characters
2. Spaces separate words
3. Each sentence delimiter (. ! ?) ends current sentence and becomes part of it
4. "e.g." is treated as three sentences: "e", "g", ""

Storage

1. Storage servers persist data in their local directories
2. Files are not replicated across storage servers
3. Round-robin assignment for new files

Design Decisions

Why 2D Linked List?

1. **Requirement Alignment**: Project explicitly requires sentence-level locking and word-level editing
2. **Dynamic Growth**: Files can grow indefinitely without reallocation
3. **Natural Parsing**: Sentence delimiters naturally create sentence boundaries
4. **Memory Efficiency**: No wasted space for fixed-size buffers
5. **Undo Implementation**: Easy to clone entire structure for version history

Alternative Considered: Array-based storage

- ✗ Fixed size limitations
- ✗ Expensive to insert/delete in middle
- ✓ Better cache locality (but not critical for this use case)

Why Trie for File Search?

1. **O(m) Complexity**: Guarantees logarithmic search time relative to filename length
2. **Prefix Search**: Can easily extend to support wildcard/prefix queries
3. **No Collisions**: Unlike hash tables, no collision handling needed
4. **Ordered Traversal**: Can iterate files alphabetically

Alternative Considered: Hash table

- ✓ Average O(1) lookup
- ✗ Worst case O(N) with collisions
- ✗ No natural ordering

Why Direct Client-SS Connection?

1. **Scalability**: Name Server doesn't become bottleneck for large file transfers
2. **Bandwidth**: Distributes network load across storage servers
3. **Latency**: Eliminates one network hop for data operations
4. **Architectural Clarity**: Separation of control (NM) and data (SS) planes

PROF

Why File-Level (Not User-Level) Undo?

1. **Collaborative Editing**: Multiple users can undo each other's changes
2. **Simpler State Management**: Don't need per-user undo stacks
3. **Storage Efficiency**: One undo stack per file instead of per user-file combination
4. **Real-World Alignment**: Most collaborative tools (Google Docs, Git) have global undo

Why Execute on Name Server?

1. **Security**: Centralized execution point easier to monitor/audit
2. **Consistency**: Same environment for all executions
3. **Access Control**: Name Server already handles permissions
4. **Architecture**: Name Server is trusted coordinator

Security Note: In production, this would need sandboxing/containerization.

Error Handling

Universal Error Codes

```
#define SUCCESS                 0
#define ERR_FILE_NOT_FOUND      1
#define ERR_PERMISSION_DENIED   2
#define ERR_FILE_EXISTS         3
#define ERR_INVALID_OPERATION   4
#define ERR_SERVER_UNAVAILABLE  5
#define ERR_NETWORK_ERROR       6
#define ERR_FILE_LOCKED         7
#define ERR_INVALID_SENTENCE    8
#define ERR_INVALID_WORD_INDEX  9
#define ERR_MEMORY_ERROR        10
#define ERR_UNKNOWN             99
```

All components use these codes consistently for clear error reporting.

Logging

All components log to `logs/<component>.log`:

```
[2024-10-20 15:30:45] [INFO] Storage Server 1 initialized at storage/ss1
[2024-10-20 15:30:46] [REQUEST] From 127.0.0.1:54321 - Operation: Client
Request
[2024-10-20 15:30:46] [RESPONSE] To 127.0.0.1:54321 - Status: Success
[2024-10-20 15:30:50] [ERROR] Failed to connect to Storage Server
```

PROF

Format: `[timestamp] [level] message`

Levels: INFO, REQUEST, RESPONSE, ERROR, DEBUG

Testing

Create Test Files

```
make test-files
```

This creates sample files in `storage/ss1/` and `storage/ss2/`.

Manual Testing Workflow

1. Start servers:

```
make run-servers
```

2. Start two clients in different terminals:

```
# Terminal 1
./client
# Enter username: alice

# Terminal 2
./client
# Enter username: bob
```

3. Test file creation (as alice):

```
nfs> CREATE shared.txt
nfs> WRITE shared.txt 1
> 0 Hello
> 1 from
> 2 alice
> 3 .
> ETIRW
```

4. Test access control (as alice):

```
nfs> ADDACCESS -W shared.txt bob
```

PROF

5. Test concurrent access (as bob):

```
nfs> READ shared.txt
nfs> WRITE shared.txt 1
> 4 Nice
> 5 to
> 6 meet
> 7 you
> 8 !
> ETIRW
```

6. Test undo (as alice):

```
nfs> UNDO shared.txt
nfs> READ shared.txt
```

7. Test streaming (as bob):

```
nfs> STREAM shared.txt
```

Known Limitations

1. **No Name Server Replication**: Single point of failure
2. **No File Replication**: Each file stored on one SS only
3. **Limited Packet Size**: MAX_BUFFER_SIZE (8KB) limits single operation size
4. **No Encryption**: All communication in plaintext
5. **No Authentication**: Username-only, no passwords
6. **No Distributed Locking**: Sentence locks only prevent concurrent edits, not true distributed locking
7. **Simplified Write**: Complex multi-sentence writes may not parse perfectly

---

Future Enhancements

1. **Security**: Add TLS/SSL encryption, password authentication
2. **Reliability**: Name Server replication, file replication across SSs
3. **Performance**: Implement true distributed caching, load balancing
4. **Features**: Directory support, file renaming, symbolic links
5. **Monitoring**: Real-time dashboard, metrics collection
6. **Scalability**: Distributed hash table for file location, sharding

---

PROF

Authors

Hot Single Threads Near You

- Course: Operating Systems and Networks
- Institution: CS3-OSN-Monsoon-2025

License

This project is submitted as coursework. All rights reserved.